Fiche d'activité — Prise en main TinyML sur Arduino Nano 33 BLE Sense Rev2

Tremplin Recherche | Parcours ciblé : IMU → ML simple → TinyML → performances → Edge Impulse

Objectif et périmètre

Découvrir de façon progressive la chaîne complète IMU → acquisition → visualisation → traitement simple → classification hors ligne → conversion/quantification → inférence embarquée → mesure des performances, puis reproduire le même type de problème avec Edge Impulse. Le but est de comprendre le fonctionnement du pipeline avant d'aborder ultérieurement un modèle de recherche plus complexe.

Progression retenue

Phase	Activités	Objectif	Résultat attendu
1 — IMU Nano	1–2	Comprendre le capteur et les signaux	Acquisition propre + features simples
2 — ML simple	3–4	Comprendre classification, dataset, entraînement et évaluation	Petit classifieur reproductible
3 — Premier TinyML	5–6	Convertir, quantifier, déployer et mesurer	Inférence INT8 temps réel sur Nano
4 — Edge Impulse	7	Comparer un workflow intégré au workflow manuel	Même problème déployé via Edge Impulse

Matériel et logiciels

- Arduino Nano 33 BLE Sense Rev2, câble USB et ordinateur.
- Arduino IDE 2, support Arduino Mbed OS Nano, bibliothèque Arduino_BMI270_BMM150.
- Google Colab / Python : NumPy, pandas, matplotlib, scikit-learn, TensorFlow/Keras.
- Pour Edge Impulse : compte Edge Impulse ; Data Forwarder si nécessaire.

Attention Rev2. La Nano 33 BLE Sense Rev2 utilise BMI270 + BMM150 et la bibliothèque Arduino_BMI270_BMM150. Les anciens exemples basés sur Arduino_LSM9DS1 doivent être adaptés.

Activité 1 — Prise en main de l'IMU et visualisation

Lire, comprendre et représenter les six axes avant tout apprentissage.

- Lire ax, ay, az, gx, gy, gz avec horodatage. Viser d'abord 50 Hz et vérifier la cadence réelle obtenue.
- Observer des situations simples : carte immobile, inclinaison lente, rotation contrôlée, mouvement plus rapide.
- Afficher les signaux dans Serial Plotter puis les enregistrer en CSV.
- Tracer dans Colab les six canaux et commenter amplitude, bruit, offset, orientation et répétabilité.

↗ Arduino_BMI270_BMM150 — bibliothèque et exemples

↗ Nano 33 BLE Sense Rev2 — documentation Arduino

Livrable / validation : Sketch d'acquisition + CSV horodaté + graphiques commentés.

Activité 2 — Traitement du signal sans ML

Introduire fenêtre, caractéristiques et décision simple avant le machine learning.

- Découper les signaux en fenêtres fixes de 1 à 2 s.
- Calculer la norme de l'accélération, moyenne, écart-type, variance et RMS.

- Créer une règle simple repos / mouvement à partir d'un seuil et commander la LED.
- Comparer les mêmes calculs sur PC et sur la Nano pour une séquence identique.

Livrable / validation : Notebook + firmware de décision par seuil + comparaison numérique PC/Nano.

Activité 3 — Premier petit ML de classification sous Colab

Comprendre comment sont représentées des données IMU et tester quelques classifieurs simples.

- Utiliser UCI Human Activity Recognition Using Smartphones (UCI HAR) comme dataset pédagogique : accéléromètre + gyroscope, 6 activités, 30 sujets, 50 Hz.
- Identifier clairement données d'entrée, labels, sujets, train/test et dimensions des tableaux.
- Tracer quelques signaux ou fenêtres représentatives et comparer visuellement plusieurs activités.
- Commencer avec les features déjà calculées et tester seulement quelques modèles simples : Logistic Regression, SVM et Random Forest.
- Comparer accuracy, macro-F1 et matrice de confusion. Le but est de comprendre le workflow, pas d'optimiser au maximum.
- Conserver le split par sujets pour éviter les fuites de données entre apprentissage et test.

↗ UCI HAR — Human Activity Recognition Using Smartphones

Livrable / validation : Notebook Colab : exploration du dataset + 2 ou 3 classifieurs + matrice de confusion.

Activité 4 — Petit modèle Keras sur les données de la Nano

Créer un problème à trois classes et entraîner un modèle neuronal volontairement simple.

- Classes proposées : immobile / mouvement lent / mouvement brusque contrôlé. Aucune chute simulée.
- Enregistrer plusieurs sessions indépendantes par classe et conserver un identifiant de session.
- Séparer train / validation / test par session AVANT le fenêtrage.
- Choisir une seule représentation pour ce premier modèle : features statistiques par fenêtre OU valeurs brutes aplaties. Rester simple.
- Entraîner un petit MLP Keras uniquement ; pas de CNN, LSTM ou Transformer à ce stade.
- Évaluer sur un test indépendant : accuracy, macro-F1, rappel par classe et matrice de confusion.
- Sauvegarder le modèle et tous les paramètres de preprocessing nécessaires au déploiement.

Livrable / validation : CSV étiquetés + notebook Keras + modèle sauvegardé + résultats de test.

Activité 5 — Premier TinyML : conversion, quantification et validation numérique

Passer du modèle Keras au modèle embarquable en vérifiant chaque étape.

- Exporter le modèle Keras FP32 vers TFLite FP32 et vérifier les prédictions sur un petit jeu de fenêtres figées.
- Effectuer ensuite une quantification post-training full INT8 avec un jeu représentatif issu du train.
- Comparer Keras FP32, TFLite FP32 et TFLite INT8 : prédictions, accuracy, macro-F1 et taille du modèle.
- Intégrer le modèle INT8 sur la Nano avec un runtime TFLite Micro / LiteRT Micro compatible et figer sa version.
- Avant l'acquisition temps réel, injecter exactement les mêmes fenêtres figées sur PC et sur MCU et comparer classes et scores déquantifiés.

Livrable / validation : Modèles .tflite FP32/INT8 + firmware de test + tableau de concordance PC/Nano.

Activité 6 — Inférence temps réel et mesures de performances

Assembler la chaîne complète et apprendre à mesurer proprement un premier système TinyML.

- Implémenter acquisition IMU, buffer de fenêtre, preprocessing, Invoke(), décodage de sortie et affichage.
- Tester les trois classes en temps réel et conserver les journaux de prédictions.
- Mesurer la taille du modèle et la Flash du firmware complet après compilation.

- Relever la tensor arena utilisée si disponible et distinguer cette valeur de la RAM totale du programme.
- Chronométrer l'inférence seule sans Serial.print dans la zone mesurée ; répéter au moins 50 fois et donner médiane + dispersion.
- Mesurer séparément le preprocessing et, si utile, le temps bout-en-bout.

Livrable / validation : Démo live + tableau accuracy / taille modèle / Flash / RAM connue / latence.

Activité 7 — Edge Impulse : refaire le même problème avec une plateforme intégrée

Identifier ce qu'Edge Impulse automatise et comparer la facilité de mise en œuvre au workflow manuel.

- Commencer par le tutoriel officiel Motion Recognition pour comprendre acquisition, labels, fenêtrage, DSP, learning block, entraînement, test et déploiement.
- Créer ensuite un projet Edge Impulse utilisant les mêmes trois classes que l'activité 4.
- Réutiliser autant que possible les mêmes données et le même découpage train/test ; sinon documenter précisément les différences.
- Importer les CSV existants ou utiliser le Data Forwarder avec un sketch Rev2 basé sur Arduino_BMI270_BMM150.
- Déployer le modèle généré sur la Nano et relever les mêmes métriques principales : accuracy, taille, RAM/Flash et latence.
- Comparer enfin workflow manuel vs Edge Impulse : préparation des données, preprocessing, entraînement, quantification, intégration, métriques et facilité de mise en œuvre.

↗ Edge Impulse — Motion recognition

↗ Edge Impulse — Data Forwarder

↗ Arduino — Edge Impulse with Nano 33 BLE Sense — utile pour la logique générale ; vérifier les différences Rev2

Livrable / validation : Projet Edge Impulse fonctionnel + courte comparaison « manuel vs Edge Impulse ».

Résultat attendu à la fin de cette première séquence

- L'étudiant sait acquérir et interpréter un signal IMU sur la Nano 33 BLE Sense Rev2.
- Il sait charger un dataset IMU public, identifier X/y et évaluer quelques classifieurs simples.
- Il sait entraîner un petit MLP Keras sur ses propres données.
- Il sait convertir et quantifier un modèle en INT8, puis vérifier son équivalence numérique avant le temps réel.
- Il sait déployer le modèle sur la Nano et mesurer des indicateurs simples de ressources et de latence.
- Il sait enfin refaire le même type d'application dans Edge Impulse et expliquer ce que la plateforme automatise.

Principe méthodologique : à chaque comparaison, ne changer qu'un élément à la fois et conserver un jeu de test figé. Le but de cette séquence est l'apprentissage du workflow TinyML, pas encore l'optimisation avancée ni la comparaison multi-MCU.